一週間で身につくC言語の基本

第５日目：構造体

シフトシステム株式会社

一週間でわかるC/C++言語

一週間でわかC言語・C++言語がオンライン講座になりました！動画と音声にってさらにわかりますくなりました!!
1講座で2つの言語が学ぶことが出来る上に、練習問題の回答もダウンロードできます。

Udemy

構造体

構造体とは

プログラムがある程度複雑になってくると、一つの概念に対して複数の変数が割り当てられることがあります。例えば学校で学生のデータベースを作る時、その学生の学生番号、名前、年齢 といったデータをひとまとめにして扱う事になります。

そういったプログラムを作る時、関連する変数がバラバラになっていると、取扱が非常に不便です。そこで便利なのが、**構造体**（こうぞうたい）という概念です。構造体とは、複数の変数をひとまとめにするものです。

たとえば、学生番号を表す整数型の変数id、名前を表す文字列name、年齢を表す整数型変数ageをひとまとめにして構造体にすると、以下のようになります。

構造体テンプレートの定義

```
struct student{
    int id;             // 学生番号
    char name[256]; // 名前
    int age;            // 年齢
};
```

このような、構造体の定義を**構造体テンプレート**と言います。**struct**が、構造体を表すキーワードであり、そのあとのstudentが構造体の名前になります。 構造体の名前は、任意につけることができます。{}の中に、ひとまとめにする変数を定義します。最後は;（セミコロン）で終了します。

この構造体を実際に使用するには、

構造体変数の定義

```
struct student data;
```

とすると、dataという名前の構造体変数を定義できます。

サンプルプログラム

では、実際に来れに代入したり、出力したりするサンプルを見てみましょう。

listex5-1：main.c

```c
#include <stdio.h>
#include <string.h>

//  学生のデータを入れる構造体
struct student{
    int id;         //  学生番号
    char name[256]; //  名前
    int age;        //  年齢
};

void main(){
    struct student data;
    data.id = 1;    //  番号を設定
    strcpy(data.name,"山田太郎");   //  名前を設定
    data.age = 18;  //  年齢を設定
    //  データの内訳を表示
    printf("学生番号:%d 名前:%s 年齢:%d¥n",data.id,data.name,data.age);
}
```

実行結果

```
学生番号:1 名前：山田太郎 年齢18
```

構造体の成分の変数のことを、**メンバ**と言います。このメンバにアクセスするには、通常以下のように行います。

構造体変数の成分へのアクセス

Programming Language

C言語 | C++言語 | Java | C#

プログラミング

1. 一週間でわかるC++言語
2. 一週間でわかるJava言語
3. 一週間でわかるC#言語
4. 一週間でわかるアルゴリズム

web関連

1. 一週間でわかるMySQL

開発環境（IDE）

1. VisualStudio2015
2. Eclipse

参考資料

1. 参考になるC言語の本
2. アスキーコード表
3. エスケープシーケンス
4. 演算子の優先順位

その他

1. コラム

リンク

1. 相互リンク
2. コンピュータリテラシ

プライバシーポリシー

1. プライバシーポリシー

```
(構造体変数名).(メンバ)
```

このサンプルでは、構造体変数はdataなので、そのメンバidへのアクセスは、間に"."(ピリオド)をつけて、"data.id"とします。このサンプルでは、13〜15行目で、 それぞれの成分に値を代入しています。先頭に"data."がついているだけで、それぞれの成分へのアクセスは、普通の変数とは変わりません。

C言語には、このほかに、構造体によくにた**共用体（きょうようたい）**という概念も存在します。興味のある方は、以下のサイトを参考にしてみてください。

→ 共用体について

構造体配列

サンプルプログラム

次は、構造体を配列にして使用する例を紹介します。以下のプログラムを実行してみてください。

listex5-2：main.c

```
 1  #include <stdio.h>
 2  #include <string.h>
 3
 4  //  学生のデータを入れる構造体
 5  struct student{
 6      int id; //  学生番号
 7      char name[256]; //  名前
 8      int age;        //  年齢
 9  };
10
11  //  構造体の名前をtypedefで定義
12  typedef struct student student_data;
13
14  void main(){
15      int i;
16      student_data data[] = {
17          { 1,"山田太郎",18 },
18          { 2,"佐藤良子",19 },
19          { 3,"太田隆",18 },
20          { 4,"中田優子",18 }
21      };
22      //  データの内訳を表示
23      for(i = 0; i < 4; i++){
24          printf("学生番号:%d 名前:%s 年齢:%d¥n",data[i].id,data[i].name,data[i].age);
25      }
26  }
```

実行結果

```
学生番号:1 名前:山田太郎 年齢:18
学生番号:2 名前:佐藤良子 年齢:19
学生番号:3 名前:太田隆 年齢:18
学生番号:4 名前:中田優子 年齢:18
```

typedef

配列変数について説明する前に、以下の処理について説明しましょう。

構造体名の変更

```
typedef struct student student_data;
```

先頭に出ている**typedef**は、既存の型に新しい名前(別名)を付けるためのキーワードで、このプログラムの場合、studentという構造体をstudent_dataというなまえに変更するという事を意味します。

再定義した構造体でのデータの定義

```
student_data s;
```

のように、先頭に"struct"キーワードをつけることなく構造体変数を定義することが可能です。

次に、構造体変数への値の代入ですが、初期値の設定の場合、16行目から21行目のように、通常変数の場合のように、{}を使って値を一度に複数定義することができます。外側の{}の中に、定義する値の数だけ、{}でメンバを定義して、間を,(コンマ)で区切ります。メンバの値の定義は、メンバの並び順に正しく代入する必要があります。

構造体のポインタ

サンプルプログラム

最後に、構造体とポインタの使い方について説明していきましょう。以下のプログラムは、listex5-2と同じ処理をポインタを使った処理に書き換えたものです。 少し長いですが、入力して実行してみてください。

listex5-3：main.c

```
1  #include <stdio.h>
2  #include <string.h>
3
4  //  学生のデータを入れる構造体
5  typedef struct{
6      int id;          //  学生番号
7      char name[256]; //  名前
8      int age;         //  年齢
9  }student_data;
10
11 //  構造体のデータを表示する関数
12 void setData(student_data*,int,char*,int);
13 void showData(student_data*);
14
15 void main(){
16     student_data data[4];
17     int i;
18     int id[] = { 1,2,3,4 };
19     char name[][256] = { "山田太郎","佐藤良子","太田隆","中田優子" };
20     int age[] = { 18,19,18,18 };
21     //  データの設定
22     for(i = 0; i < 4; i++){
23         setData(&data[i],id[i],name[i],age[i]);
24     }
25     //  データの内訳を表示
26     for(i = 0; i < 4; i++){
27         showData(&data[i]);
28     }
29     return;
30 }
31 //  データのセット
32 void setData(student_data* data,int id,char* name,int age){
33     data->id = id;               //  idのコピー
34     strcpy(data->name,name); //  名前のコピー
35     data->age = age;         //  年齢のコピー
36 }
37 //  データの表示
38 void showData(student_data* data){
39     printf("学生番号:%d 名前:%s 年齢:%d¥n",data->id,data->name,data->age);
40 }
```

実行結果は、listex5-1と同じなので省略します。構造体ポインタに関する説明をする前に、5～9行目を見てください。

構造体テンプレートの定義と、名前の変更の一括処理

```
typedef struct{
    int id;            // 学生番号
    char name[256]; // 名前
    int age;           // 年齢
}student_data;
```

アロー演算子

この処理を行うと、構造体テンプレートの定義と、名前の変更が同時にできます。したがって、16行目のように、"struct"キーワード抜きで構造体変数を定義できます。

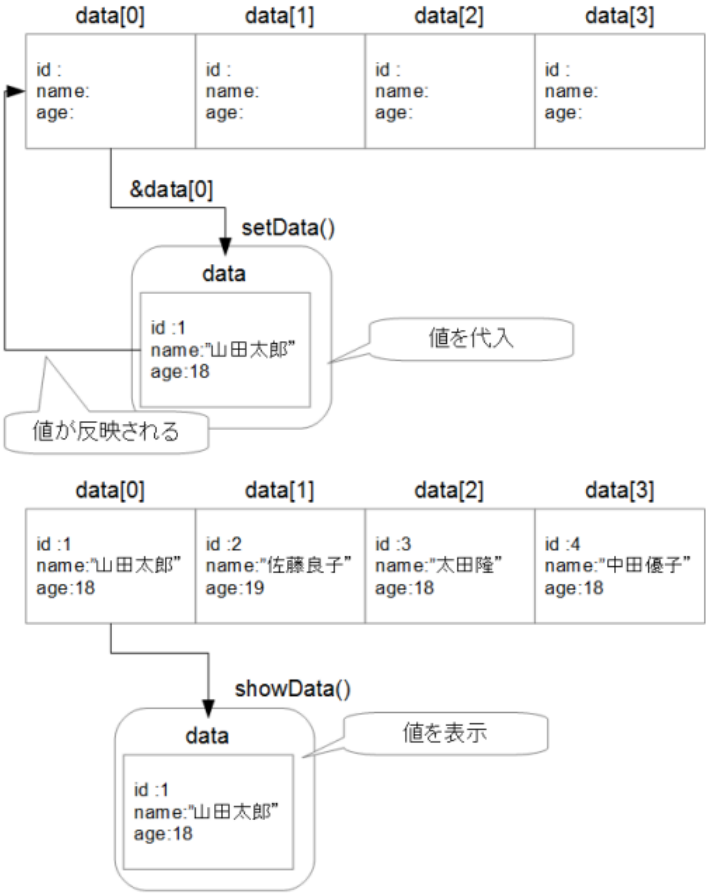
次に、本題であるポインタの構造体について説明しましょう。通常の構造体では、メンバにアクセスするのに"."を用いますが、ポインタの場合は、"**->**"（アロー演算子)を用います。(表5-1)

表5-1.通常の構造体とポインタの構造体(student_data)

	通常の構造体	構造体のポインタ
定義	student_data data	student_data* pData
メンバ	data.id data.name data.age	data->id data->name data->age

showData()関数およびshowData()関数では、ポインタの形式でデータが渡ってきます。そのため、33～35行目、もしくは39行目では、アロー演算子が用いられています。ここで、値の表示およびデータの代入が行われています。(図5-1)

図5-1.setData()/showData()の働き



通常、構造体を関数の引数として渡す場合は、このサンプルのようにアドレスを渡すのが普通です。では、いったいなぜそのようなことをするのでしょう？次でそのことを詳しく説明しましょう。

ポインタ渡しとデータ渡し

サンプルプログラム

ポインタ渡しとデータ渡しの違いを理解するために、まずは以下のプログラムを入力・実行してみてください。

listex5-4：main.c

```c
#include <stdio.h>

//  データを入れる構造体
typedef struct{
    int a;
    double d;
}num_data;

//  二種類の値設定関数
void dealData1(num_data data);      //  値渡し
void dealData2(num_data* pData);    //  ポインタ渡し

void main(){
    num_data n1 = { 1, 1.2f },n2 = { 1, 1.2f };
    printf("n1のアドレス:0x%x n2のアドレス:0x%x\n",&n1,&n2);
    dealData1(n1);
    dealData2(&n2);
    printf("n1.a = %d n2.d = %f¥n",n1.a,n1.d);
    printf("n2.a = %d n2.d = %f¥n",n2.a,n2.d);
}

void dealData1(num_data data)
{
    printf("a=%d f=%f\n",data.a,data.d);
    printf("dealData1にわたってきたデータのアドレス:0x%x¥n",&data);
    //  値の変更
    data.a = 2;
    data.d = 2.4;
}
void dealData2(num_data* pData)
{
    printf("a=%d f=%f\n",pData->a,pData->d);
    printf("dealData2にわたってきたデータのアドレス:0x%x¥n",pData);
    //  値の変更
    pData->a = 2;
    pData->d = 2.4;
}
```

実行結果

```
n1のアドレス:0x2c728 n2のアドレス:0x2cf710
a=1 f=1.200000
dealData1にあたってきたデータのアドレス:0x2cf630
a=1 f=1.200000
dealData2にあたってきたデータのアドレス:0x0x2cf710
n1.a = 1 n2.d = 1.200000
n1.a = 2 n2.d = 2.400000
```

実行結果より、値渡しの場合は、dealData1()関数に渡った引数のアドレスはもとの数とは異なります。ポインタ渡しのdealData2()の場合は、同じアドレスとなります。 そのため、showData1()では、値の変更を行っても、main()には反映されませんが、showData2()の場合は、アドレスが同じであることから、値の変更が反映されます。

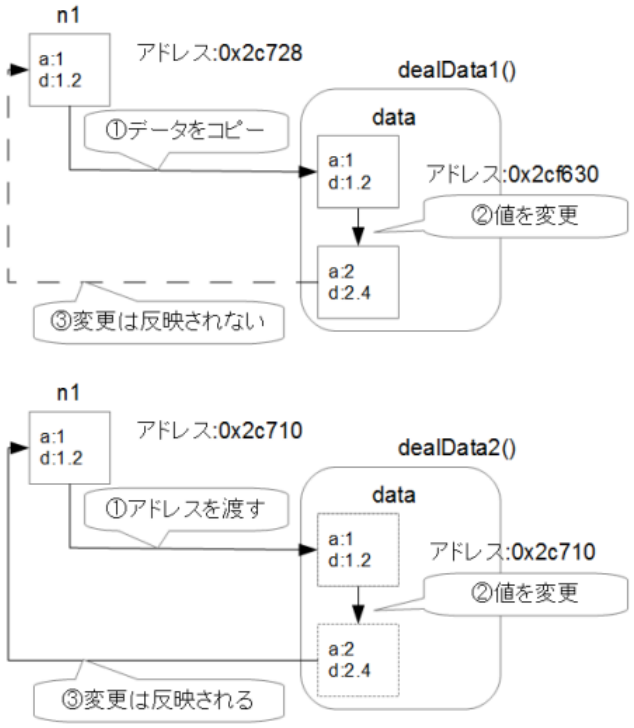
これが、データ渡しとポインタ渡しの違いです。

データ渡しの問題点

ポインタを渡す理由は二つあります。一つは、通常、構造体のデータのサイズは大きくなる傾向があり、引数としてそのままの値を渡すと、スタック領域を圧迫してしまったり、データのコピーという無駄な処理が起こり、二重の意味でリソースを無駄にしてしまうからです。

そして、もうひとつの理由は、ポインタ渡しであれば、関数の中で値の設定などが出来るからです。値渡しでは、前述のようにコピーが発生する上に、引数として渡ってきた構造体のデータを変更しても、呼び出しもとの値に反映されません。(図5-2)

図5-2.構造体の値渡しと参照渡し



以上のような理由から、構造体を関数の引数として渡す場合は、ポインタ渡しを用いるのが普通なのです。

◀ BACK ▲ TOP NEXT ▶

Copyright © shift-system.jp All Rights Reserved.